Livre 13

Le genèsis humain de l'automatisation

Pourquoi les humains automatisent : les origines profondes

Partie A

« Automatiser sans comprendre pourquoi, c'est courir sans savoir vers où. »

Images et couverture par Soléane Bélanger
Collection ToTAL — Réconciliation · 21 livres
ORIGINALIUM

Collection ToTAL — Réconciliation · 21 livres

Livre 13 — Le genèsis humain de l'automatisation
Partie A — Pourquoi les humains automatisent : les origines profondes

Publié par ORIGINALIUM.

Tous les cas, anecdotes et noms d'organisations mentionnés dans cet ouvrage sont soit fictifs, soit anonymisés afin de protéger l'identité des parties concernées.

Toute ressemblance avec des situations réelles est soit fortuite, soit délibérément généralisée à des fins pédagogiques.

La collection ToTAL — Réconciliation est une œuvre de formation professionnelle destinée aux équipes de développement logiciel, aux gestionnaires de produit, aux professionnels de la qualité et à toute personne engagée dans la transformation numérique responsable.

Images et couverture par Soléane Bélanger.

Reproduit avec permission. Toute reproduction partielle ou totale sans autorisation écrite de l'éditeur est interdite.

Table des matières — Partie A

Épigraphe

Introduction du Livre 13 — L'automatisation et la question que l'industrie n'a jamais vraiment résolue

Chapitre 1 — L'automatisation : une pulsion humaine ancestrale

1.1 L'automatisation avant l'informatique

1.2 Les cinq motivations profondes derrière l'automatisation

1.3 Le piège de la Motivation 5 — L'oncle MIDI et la curiosité pure

1.4 La question fondamentale avant d'automatiser

1.5 Terrain — L'éditeur de logiciel ERP comptable : automatiser les mauvais tests

1.6 Scène DREAM-TEAM — La question de Théo

À retenir · Symptômes · Questions de réflexion · Signal de vigilance

Chapitre 2 — Ce qui doit et ne doit pas être automatisé dans le SDLC réconcilié

2.1 Tout n'est pas automatisable — et tout n'a pas à l'être

2.2 Ce qui doit être automatisé en zone FORGE

2.3 Ce qui ne doit pas être automatisé en zone SENSE

2.4 Ce qui ne doit pas être automatisé en zone CARE

2.5 Le test exploratoire — ce qui ne sera jamais automatisé

2.6 Terrain — La Mission Faucon et le bug invisible à la machine

2.7 Scène DREAM-TEAM — La carte d'automatisation d'ORIGINALIUM

À retenir · Symptômes · Questions de réflexion · Signal de vigilance

Chapitre 3 — Les quatre pièges de l'automatisation mal ciblée

3.1 Piège 1 — Automatiser le mauvais problème

3.2 Piège 2 — La dette d'automatisation

3.3 Piège 3 — L'atrophie des compétences

3.4 Piège 4 — La fausse sécurité

3.5 Terrain — Le studio de développement de jeux vidéo et le DLC catastrophique

À retenir · Symptômes · Questions de réflexion · Signal de vigilance

Chapitre 4 — ONETOOL : la tentation de l'outil miracle

4.1 L'histoire de ONETOOL

4.2 La loi de la dépendance à l'outil

4.3 Comment évaluer un outil d'automatisation

4.4 Scène DREAM-TEAM — La tentation ONETOOL

À retenir · Symptômes · Questions de réflexion · Signal de vigilance

Conclusion de la Partie A — Ce que nous avons compris, et ce qui attend en Partie B

Quatrième de couverture

« Automatiser sans comprendre pourquoi,

c'est courir sans savoir vers où. »

— Fil conducteur du Livre 13, Collection ToTAL — Réconciliation

---

INTRODUCTION

L'automatisation et la question que l'industrie n'a jamais vraiment résolue

Livre 13 est peut-être le livre le plus philosophique de la collection ToTAL — Réconciliation. Il ne traite pas de pipelines CI/CD, ni de scripts de test, ni d'outils d'intégration continue. Il traite d'une question plus fondamentale, plus ancienne et, paradoxalement, plus urgente que toutes celles que nous avons abordées jusqu'ici : ***pourquoi automatisons-nous ?***

Si cette question vous semble évidente — si vous pensez y répondre d'emblée en deux phrases —, nous vous invitons à suspendre ce jugement pendant quelques pages. Car dans l'industrie du logiciel, la réponse à cette question est presque toujours plus floue qu'il n'y paraît. Et c'est précisément de cette flou que naissent les projets d'automatisation qui échouent, les pipelines qui deviennent ingérables, les équipes qui perdent leurs compétences fondamentales, et les systèmes qui passent tous leurs tests mais s'effondrent en production.

Ce livre est né d'une conviction simple : l'industrie du logiciel n'a jamais vraiment résolu la question de l'automatisation. Elle a résolu *comment* automatiser — et elle l'a fait brillamment. Des générations d'ingénieurs ont développé des outils, des frameworks, des paradigmes, des philosophies entières autour de l'automatisation. Mais la question *pourquoi* automatiser, et surtout *pourquoi pas*, a été étrangement négligée. Comme si l'évidence de la réponse rendait la question inutile.

Ce livre prend le problème par son origine. Nous commençons non pas avec des frameworks, mais avec une cave.

L'anecdote fondatrice : l'oncle MIDI

Quelque part dans une maison de banlieue tranquille, un homme — appelons-le l'oncle MIDI — descend chaque soir dans sa cave. Il est ingénieur de profession. Passionné de musique depuis l'enfance, il ne joue d'aucun instrument — du moins pas assez bien pour en être fier. Mais il a découvert quelque chose de merveilleux : le protocole MIDI, et avec lui, la possibilité d'automatiser entièrement la production musicale.

Pendant des mois, l'oncle MIDI programme. Il code des séquences rythmiques d'une précision métronomique, des progressions harmoniques sophistiquées, des variations dynamiques que ses doigts n'auraient jamais pu reproduire sur un clavier. Il empile couche après couche. Il optimise. Il itère. Son système MIDI est, sur le plan technique, un chef-d'œuvre d'ingénierie. Les séquences sont parfaitement synchronisées. Les timbres sont soigneusement choisis. Les transitions sont fluides. Le tout tourne de manière impeccable, sans jamais faire d'erreur, sans jamais se fatiguer, sans jamais dérailler.

Un dimanche après-midi, il invite sa famille à écouter. Les enfants, la femme, quelques cousins. Il descend dans la cave, charge le projet, appuie sur Play. La musique emplit la pièce. Elle est belle. Elle est riche. Elle est techniquement irréprochable.

Un long silence s'installe à la fin du morceau. Puis, avec cette franchise désarmante propre aux enfants, l'un des neveux pose la question :

> **Le neveu :** *C'est beau, Oncle. Mais c'est toi qui joues ?*
>
> **L'oncle MIDI :** *Non... c'est l'ordinateur.*

Nouveau silence. Celui-là, plus lourd.

L'oncle MIDI regarde les visages autour de lui. La politesse est là. L'admiration technique, peut-être. Mais quelque chose manque — quelque chose d'indéfinissable mais d'essentiel. Ce quelque chose, il mettra quelques jours à le nommer. Et quand il le nommera, la révélation sera foudroyante : **il a automatisé la musique, mais pas la joie de la faire.** Il a automatisé l'output — le son parfait, la séquence précise — mais pas l'expérience. Pas la connexion. Pas la présence humaine dans l'acte de créer.

Il n'avait pas tort d'automatiser. Son système était élégant. Mais il avait omis de se poser la vraie question : *que cherchait-il vraiment à produire ?* La musique parfaite ? Ou le moment de jouer pour ceux qu'il aimait ? Ces deux intentions mènent à des architectures radicalement différentes — et à des décisions d'automatisation radicalement différentes.

C'est la question centrale de ce livre 13.

La place du Livre 13 dans la collection ToTAL

La collection ToTAL — Réconciliation compte 21 livres. Elle a commencé par les fondations : la méthode, les spécifications IF…THEN…, la qualimétrie, les six phases du SDLC réconcilié — **ÉLICITER · DÉFINIR · PLANIFIER · IMPLÉMENTER · DIFFUSER · CONFIRMER** —, les trois zones — **SENSE · FORGE · CARE** —, la grande boucle CONFIRMER → ÉLICITER. Elle a progressivement abordé des dimensions plus complexes : les post-mortems, l'investigation des anomalies, le test exploratoire, l'intégration de l'intelligence artificielle comme outil d'assistance.

Le Livre 13 complète cette trajectoire en posant la question ultime : dans tout ce que nous avons construit, **quel est le rôle de l'humain ?** Et comment nous assurer qu'il reste au centre — non pas comme un opérateur qui supervise des machines, mais comme un agent de jugement, de sens et de décision irremplaçable ?

La chaîne fondamentale de la collection — **Clarté → qualité → confiance → excellence durable** — n'est possible que si chaque maillon est tenu par des humains qui comprennent ce qu'ils font et pourquoi. L'automatisation, bien ciblée, renforce cette chaîne. Mal ciblée, elle la ronge de l'intérieur.

Ce que vous trouverez dans ce livre

Le Livre 13 est structuré en deux parties. La **Partie A** — que vous lisez en ce moment — examine les *origines profondes* de l'automatisation : pourquoi les humains automatisent, ce qui devrait et ne devrait pas l'être, les pièges les plus communs, et la tentation de l'outil miracle. La **Partie B** approfondira la mécanique de l'automatisation responsable : comment construire une stratégie d'automatisation ancrée dans le SDLC réconcilié, comment maintenir les compétences humaines en parallèle, et comment évaluer en continu si une automatisation remplit toujours son intention d'origine.

Tout au long de ce livre, vous retrouverez la DREAM-TEAM d'ORIGINALIUM : **Rania** (tech lead), **Théo** (Product Owner) et **Karim** (QA). Leurs dialogues ne sont pas des exemples parfaits à imiter — ce sont des conversations honnêtes, parfois inconfortables, qui reflètent ce que vivent réellement les équipes quand elles se posent sérieusement la question de l'automatisation.

Chaque chapitre se termine par quatre encadrés structurés : **À retenir**, **Symptômes à observer**, **Questions de réflexion**, et **Signal de vigilance**. Ces encadrés sont conçus pour être utilisés en rétrospective d'équipe, en revue de sprint, ou simplement comme points d'ancrage personnels.

Une promesse et une mise en garde

Ce livre ne vous dira pas quoi automatiser. Il ne vous donnera pas de liste universelle des outils à adopter. Il ne vous proposera pas de recette en dix étapes vers le pipeline parfait. Ce qu'il vous offrira, c'est quelque chose de plus durable : un cadre de réflexion, un vocabulaire de décision, et une posture de discernement

que vous pourrez appliquer à chaque choix d'automatisation que vous ferez dans votre carrière.

La mise en garde est simple : si à la fin de ce livre, vous pensez que l'automatisation est soit toujours bonne, soit toujours mauvaise, nous n'aurons pas réussi. L'automatisation n'est ni un idéal ni un danger — c'est un outil. Et comme tout outil, sa valeur est entièrement déterminée par la compréhension et l'intention de celui qui le tient.

> *« La valeur d'un outil n'est pas dans sa puissance. Elle est dans la précision de l'intention qui le guide. »*
>
> — Principe fondateur de la collection ToTAL — Réconciliation

Descendons maintenant dans la cave de l'oncle MIDI — et posons-nous les vraies questions.

CHAPITRE 1

L'automatisation : une pulsion humaine ancestrale

Avant de parler de pipelines CI/CD, de scripts Python ou de robots de déploiement, il faut remonter beaucoup plus loin. L'automatisation n'est pas un artefact de l'ère numérique. Elle est une constante de l'histoire humaine — une réponse instinctive à la condition fondamentale d'être un être intelligent dans un monde qui exige de la répétition.

1.1 L'automatisation avant l'informatique

Revenons deux mille ans en arrière. Dans une vallée méditerranéenne, un meunier observe l'eau d'un ruisseau qui coule sans relâche. Il se rend compte que cette eau travaille, d'une certaine manière — elle porte une énergie cinétique constante, sans se fatiguer, sans demander de salaire, sans avoir besoin de pauses. Il construit une roue. La roue tourne. Le grain est moulu. L'humain est libéré de l'une des tâches les plus épuisantes de l'existence rurale. Le moulin à eau est né.

Ce n'est pas seulement un exploit d'ingénierie. C'est un acte philosophique. Le meunier a reconnu qu'une tâche répétitive, physiquement identique, pouvait être confiée à un système qui ne se lasserait jamais. Il a compris qu'il existait une frontière entre ce que l'humain doit faire et ce qu'il peut déléguer. Et il a agi en conséquence.

Cette intuition est restée constante à travers les siècles, prenant des formes de plus en plus sophistiquées. L'horloge mécanique du Moyen Âge automatise la mesure du temps — une tâche qui exigeait auparavant une présence humaine continue pour surveiller le sablier ou le gnomon. Elle libère l'attention humaine et standardise la coordination sociale dans les villes et les monastères. L'automatisation du temps est, dans les faits, l'une des révolutions les plus profondes de l'histoire occidentale : elle rend possible la coordination à grande échelle, le commerce régulé, la planification longue distance.

Au 18e siècle, Joseph-Marie Jacquard fait quelque chose de révolutionnaire : il introduit des cartes perforées dans le métier à tisser. Ces cartes, disposées en séquence, dictent au métier le motif qu'il doit produire — ligne après ligne, avec une régularité et une précision impossibles pour les doigts humains les plus habiles. Le métier Jacquard est, en quelque sorte, le premier dispositif programmé de l'histoire industrielle. Il ne produit pas seulement du tissu : il produit de la complexité à l'échelle, de manière reproductible. Il préfigure directement la carte perforée d'Herman Hollerith, puis les programmes informatiques du 20e siècle.

Puis vient Henry Ford, au début du 20e siècle, avec la chaîne de montage. L'idée est simple en apparence : si l'on décompose la fabrication d'une automobile en tâches élémentaires et répétitives, et que l'on fixe chaque ouvrier à une seule tâche, on peut produire des automobiles à une vitesse et un coût inimaginables dans l'artisanat traditionnel. Mais derrière cette simplicité apparente se cache une révolution conceptuelle : la chaîne de montage automatise non pas une tâche, mais un processus entier. Elle transforme le travail humain en composant d'un système, au lieu d'en faire le moteur.

Ce moment est crucial pour notre réflexion, parce qu'il révèle pour la première fois une tension que nous retrouverons tout au long de ce livre : **quand on automatise un processus, on optimise l'output — mais on transforme aussi l'expérience de ceux qui y participent.** Les ouvriers de la chaîne Ford produisaient plus, gagnaient mieux, vivaient dans des conditions moins dangereuses que les artisans d'antan — mais beaucoup d'entre eux décrirent leur travail comme monotone, aliénant, dépourvu de sens. La question de l'oncle MIDI se posait déjà à Detroit, en 1913 : qu'avait-on vraiment automatisé ? Et vers quoi ce gain de temps libérait-il réellement l'humain ?

Ces exemples — le moulin à eau, l'horloge mécanique, le métier Jacquard, la chaîne de montage — ne sont pas simplement des précédents historiques anecdotiques. Ils établissent un pattern fondamental que nous reconnaissons dans chaque projet d'automatisation logicielle contemporain. Les motivations sont les mêmes. Les tensions sont les mêmes. Les erreurs aussi.

Comprendre d'où vient l'automatisation, c'est comprendre pourquoi nous l'adoptons si facilement, si instinctivement — et pourquoi cette facilité peut devenir un piège.

1.2 Les cinq motivations profondes derrière l'automatisation

Après des décennies d'observation des équipes logicielles — dans des environnements aussi variés qu'une startup fintech en pleine expansion, un

studio de développement de jeux vidéo, une grande entreprise technologique spécialisée en systèmes embarqués temps réel, ou une grande institution financière coopérative —, il est possible de distinguer cinq motivations fondamentales qui poussent les humains à automatiser. Ces motivations sont universelles. Elles transcendent les technologies, les langages de programmation, les secteurs d'activité. Et chacune d'elles est, dans son intention initiale, absolument légitime.

Motivation 1

LA LIBÉRATION — Se libérer des tâches répétitives pour faire des choses plus intéressantes

C'est la motivation la plus noble et la plus ancienne. Le meunier qui construit une roue à eau ne cherche pas à devenir paresseux — il cherche à consacrer son énergie à ce que seul un humain peut faire : juger de la qualité de la farine, gérer la relation avec les clients, innover dans ses techniques. La libération des tâches répétitives est la promesse fondamentale de toute automatisation digne de ce nom.

Motivation 2

LA PRÉCISION — Atteindre un niveau de précision impossible pour l'humain sur la durée

Un testeur humain qui exécute le même scénario de test pour la huit-centième fois est physiologiquement incapable de maintenir le même niveau d'attention qu'à la première exécution. Ce n'est pas un manque de professionnalisme — c'est de la biologie. Un test automatisé, lui, exécute le scénario de la même manière à la première et à la huit-millième itération. La précision par l'automatisation n'est pas une critique du travail humain ; c'est une

reconnaissance honnête de ses limites sur la durée.

Motivation 3

L'ÉCHELLE — Faire à 1 000 ce qui ne peut se faire qu'à 1 à la main

Certains problèmes sont intrinsèquement des problèmes d'échelle. Une firme conseil nationale en transformation numérique qui gère des centaines de déploiements par mois ne peut pas confier chaque déploiement à un opérateur humain qui suit une liste de vérification manuelle. L'échelle change la nature du problème. Ce qui était gérable à dix est ingérable à mille. L'automatisation à l'échelle n'est pas un luxe — c'est souvent une nécessité opérationnelle absolue.

Motivation 4

LA SÉCURITÉ — Éliminer les tâches dangereuses ou épuisantes

Dans le logiciel, le danger est rarement physique — mais l'épuisement cognitif est réel. Un professionnel de la qualité qui surveille manuellement un environnement de production pendant six heures de nuit, cherchant des anomalies dans des logs de plusieurs gigaoctets, est exposé à un risque d'erreur critique directement proportionnel à sa fatigue. Automatiser la surveillance, l'alerte, le filtrage des signaux — c'est une décision de sécurité humaine, pas seulement d'efficacité opérationnelle.

Motivation 5

LA CURIOSITÉ — Parce que c'est techniquement fascinant

C'est la motivation la plus honnête, la plus répandue dans l'industrie logicielle — et la plus rarement avouée. L'ingénieur qui automatise son pipeline de déploiement le samedi matin, non pas parce que l'équipe en a besoin, mais parce que le problème lui semble élégant et le challenge stimulant. La curiosité pure : *je peux, donc j'explore*. Elle est à l'origine de certaines des plus grandes innovations de l'histoire de l'informatique. Et elle peut aussi mener à des catastrophes spectaculaires.

Chaque motivation est noble dans son intention. Aucune n'est intrinsèquement mauvaise. Mais chacune peut devenir un piège précisément lorsqu'elle n'est pas questionnée — lorsqu'elle agit en silence, dans l'ombre de la décision d'automatiser, sans jamais être nommée.

La première règle d'or du SDLC réconcilié en matière d'automatisation est donc cette : **identifier sa motivation avant de commencer à construire.** Non pas pour la juger, mais pour savoir ce qu'on optimise réellement — et ce qu'on risque de sacrifier.

1.3 Le piège de la Motivation 5 — L'oncle MIDI et la curiosité pure

Revenons à l'oncle MIDI. Quelle était sa motivation ? En y réfléchissant honnêtement, il est difficile de la classer simplement dans la catégorie « Libération » ou « Précision ». L'oncle MIDI ne cherchait pas à gagner du temps — il avait tout son temps le soir et le week-end. Il ne cherchait pas à atteindre une précision impossible — il ne prétendait pas être un interprète professionnel. Il ne cherchait pas à produire à l'échelle — une seule écoute en famille suffisait à son ambition.

Non. L'oncle MIDI automatisait parce que le problème était fascinant. Parce que programmer des séquences MIDI, c'est élégant. Parce que chaque nouvelle

couche ajoutée était une petite victoire intellectuelle. Parce que la technique elle-même était une source de joie — indépendamment du résultat final.

C'est la Motivation 5 à l'état pur. Et il n'y a rien de répréhensible là-dedans. La curiosité technique est un carburant précieux. Elle produit des innovations que la planification rationnelle n'aurait jamais générées. Elle est à l'origine du protocole MIDI lui-même, d'ailleurs.

Le piège, cependant, est subtil. Quand la curiosité devient la motivation principale, on optimise pour le problème qui nous fascine — pas pour le problème que l'utilisateur final a. On construit une solution élégante à un problème mal posé. Et on peut aller très loin dans cette direction sans jamais sentir qu'on se trompe de chemin, parce que chaque étape est techniquement satisfaisante.

L'oncle MIDI a construit un système impeccable. Il a optimisé pour la perfection musicale objective — tempo, harmonie, timbre, dynamique. Il a atteint son objectif technique. Ce qu'il n'a pas réalisé, c'est que ce n'était pas le bon objectif. L'objectif réel — celui qui aurait rendu ses proches émerveillés plutôt que polis — était de *les faire vivre quelque chose*. Et pour les faire vivre quelque chose, sa présence, son effort visible, sa vulnérabilité dans l'acte de jouer, étaient peut-être plus importants que la perfection du résultat.

Dans l'industrie du logiciel, cette histoire se répète constamment. Une équipe passe trois mois à construire un framework de test automatisé sur mesure — parce que le problème architectural était stimulant — alors qu'un outil standard aurait répondu à 90% des besoins en deux semaines. Une autre équipe automatise son processus de génération de rapports avec une sophistication remarquable, sans jamais vérifier si les destinataires des rapports les lisaient réellement. Un lead développeur consacre ses week-ends à optimiser le temps d'exécution du pipeline CI de 8 minutes à 5 minutes 30 — une réduction impressionnante — pendant que l'équipe attend, elle, la résolution d'un bug de production depuis dix jours.

La curiosité pure n'est pas une erreur. C'est une ressource. Mais comme toute ressource, elle doit être orientée. Elle doit être mise en dialogue avec les autres motivations — et surtout avec la question fondamentale que nous abordons dans la prochaine section.

> *« L'ingénieur le plus dangereux n'est pas celui qui ne sait pas. C'est celui qui sait très bien ce qu'il fait — mais n'a jamais demandé pourquoi il le fait. »*
>
> — Réflexion issue de la collection ToTAL — Réconciliation

1.4 La question fondamentale avant d'automatiser

Il existe une question — une seule, mais décisive — que toute équipe devrait se poser avant de commencer tout projet d'automatisation. Cette question n'est pas technique. Elle n'est pas budgétaire. Elle n'est pas organisationnelle. Elle est intentionnelle :

> *« Qu'est-ce que cette automatisation libère — et vers quoi ? »*

La formulation est délibérément double. La première partie — *qu'est-ce qu'elle libère* — identifie la contrainte qu'on veut supprimer. La seconde — *vers quoi* — identifie ce qu'on veut rendre possible grâce à cette libération. Et c'est la seconde partie qui est presque toujours oubliée.

Prenons un exemple concret : automatiser les tests de régression. La première partie est claire — on libère l'équipe QA de l'exécution manuelle répétitive de cas de test connus. Mais vers quoi ? Si la réponse est vague — « gagner du temps », « être plus efficace » — alors l'automatisation risque de libérer du temps qui sera simplement rempli par d'autres activités peu stratégiques, ou pire, par plus de tests automatisés sans réflexion sur leur pertinence.

Mais si la réponse est précise — « libérer les membres de l'équipe QA pour qu'ils puissent consacrer 40% de leur temps au test exploratoire sur les nouvelles fonctionnalités à haut risque » — alors on a un objectif clair. On sait non seulement ce qu'on automatise, mais ce que l'automatisation rend possible. Et cette clarté change tout : elle détermine quels tests méritent d'être automatisés en priorité, à quel degré de sophistication, avec quel niveau de maintenance tolérable.

La règle ToTAL en matière d'automatisation est donc la suivante :

Règle ToTAL — Automatisation

Toute décision d'automatiser doit pouvoir se formuler ainsi : « Nous automatisons [X] pour libérer [qui] de [quelle contrainte], afin qu'il/elle puisse consacrer cette capacité à [activité à plus haute valeur]. »

Si vous ne pouvez pas compléter cette phrase avec précision, l'automatisation est probablement prématurée — ou guidée par la Motivation 5 sans garde-fous.

Deux autres dimensions méritent d'être examinées lors de cette question préalable :

La dimension de maintenance. Toute automatisation génère une dette de maintenance. Un test automatisé qui n'est jamais mis à jour finit par tester quelque chose qui n'existe plus — ou pire, par masquer des bugs réels sous des faux positifs. La question n'est pas seulement « que libère cette automatisation ? » mais aussi « combien de capacité investit-on pour la maintenir ? » et « qui le fait ? »

La dimension de compétence. Certaines automatisations libèrent des tâches que l'équipe devrait continuer à savoir faire — parce que le jour où l'outil tombera en panne, ou changera de paradigme, ou sera abandonné par son éditeur,

l'équipe devra retrouver cette compétence. L'automatisation totale d'une compétence critique est une forme de dette de compétence, aussi dangereuse que la dette technique. Nous y reviendrons en détail au Chapitre 3.

1.5 Terrain — L'éditeur de logiciel ERP comptable : automatiser les mauvais tests

Cas terrain — Éditeur de logiciel ERP comptable

Un éditeur de logiciel ERP comptable dont le produit est utilisé par des milliers de PME décide, un jour, de faire de l'automatisation des tests une priorité stratégique. L'intention est bonne : l'équipe QA passe l'essentiel de ses sprints à exécuter des cas de régression manuels qui prennent des jours entiers. La direction veut libérer cette capacité pour accélérer les cycles de livraison.

En six mois, l'équipe construit un portfolio impressionnant : plus de 2 400 tests automatisés. Le taux de couverture des cas de test atteint 83%. Les métriques sont belles. Les dashboards sont verts. La direction applaudit.

Mais quelqu'un fait une observation gênante lors d'une rétrospective : les 2 400 tests automatisés couvrent presque exclusivement les flux heureux des modules comptables les plus stables. Les modules de gestion des taxes provinciales, récemment modifiés pour respecter de nouvelles réglementations, ne sont pratiquement pas couverts. Les cas limites des modules de paie — les plus sensibles, ceux qui génèrent les bugs les plus coûteux — ne sont pas testés du tout en automatisé.

Pourquoi ? Parce que ces cas sont *difficiles à automatiser*. Ils demandent une configuration d'état complexe, des données de test élaborées, une compréhension fine des règles métier. Les membres de l'équipe QA avaient naturellement automatisé les tests les plus simples — ceux qui fonctionnaient toujours depuis des années — parce que c'était techniquement plus

accessible et immédiatement satisfaisant.

Résultat : 83% de couverture sur les 20% les moins risqués. Et 0% de couverture sur les 20% les plus risqués. La release suivante introduit un bug de calcul de taxes qui affecte des centaines de clients. Le bug passe à travers tous les 2 400 tests automatisés sans déclencher une seule alerte.

Ce que cette anecdote enseigne : L'automatisation doit être guidée par le risque, pas par la facilité technique. Un test facile à automatiser n'est pas nécessairement un test précieux à automatiser. La question « qu'est-ce que cette automatisation libère, et vers quoi ? » aurait dû orienter l'effort vers les zones à plus haut risque — même si elles étaient techniquement plus exigeantes.

1.6 Scène DREAM-TEAM — La question de Théo

Scène DREAM-TEAM — ORIGINALIUM · Session de planification de sprint

L'équipe est réunie dans la salle de réunion principale d'ORIGINALIUM. Karim a passé les quinze premières minutes à présenter son plan d'automatisation pour le trimestre à venir : 180 nouveaux cas de test automatisés, une refonte du pipeline de régression, et l'intégration d'un nouveau framework de test end-to-end particulièrement populaire dans la communauté.

Rania pose des questions techniques — sur l'architecture du framework, sur la compatibilité avec l'environnement d'intégration continue existant, sur les dépendances de maintenance. Les réponses de Karim sont solides. Il a fait son travail de recherche.

Théo, lui, écoute. Il n'a pas encore parlé. Puis, au moment où Karim passe à la diapositive des « prochaines étapes », Théo lève la main.

> **Théo :** *Avant de parler du « quoi », j'ai besoin de comprendre le « pourquoi ». Qu'est-ce que ça libère, tout ça ?*
>
> **Karim :** *On libère du temps sur la régression manuelle. On gagne probablement deux jours par sprint.*
>
> **Théo :** *Deux jours. Pour faire quoi, exactement ?*
>
> **Karim :** *Pour… faire d'autres tests. Des tests plus avancés.*
>
> **Théo :** *Lesquels ?*

Silence. Karim regarde ses notes. Rania regarde Karim. Personne ne dit rien pendant six secondes — une éternité dans une salle de réunion.

> **Karim :** *Honnêtement, Théo ? Je pense que j'automatise parce que c'est plus intéressant à construire que de documenter les cas manuels. C'est là où j'ai envie de mettre mon énergie ce trimestre.*

Moment de silence. Puis Rania rit doucement.

> **Rania :** *C'est la réponse la plus honnête que j'ai entendue sur l'automatisation depuis deux ans. Et maintenant qu'on sait ça, on peut avoir une vraie conversation.*

> **Théo :** *Exactement. L'intention de Karim est valide — construire quelque chose de sophistiqué, monter en compétences, livrer quelque chose dont il sera fier. Ce n'est pas rien. Mais si c'est l'objectif principal, ça doit se savoir. Ça change ce qu'on automatise, dans quel ordre, et avec quel budget de temps de maintenance qu'on est prêts à accepter ensuite.*

Ce dialogue marque un tournant dans la culture QA d'ORIGINALIUM. Désormais, chaque initiative d'automatisation commence par la même question : *« Qu'est-ce que ça libère, et vers quoi ? »* Et chaque réponse — même honnêtement motivée par la curiosité — est reçue sans jugement, mais orientée avec intention.

À retenir — Chapitre 1

- L'automatisation est une pulsion humaine ancestrale, aussi vieille que le moulin à eau. Elle est fondamentalement noble dans son intention.
- Il existe cinq motivations profondes : la Libération, la Précision, l'Échelle, la Sécurité, et la Curiosité. Chacune est légitime — chacune peut devenir un piège si elle n'est pas nommée.
- La Motivation 5 (Curiosité) est la plus répandue dans l'industrie logicielle et la plus rarement avouée. Elle produit des automatisations techniquement élégantes qui répondent parfois au mauvais problème.
- L'oncle MIDI a automatisé l'output — le son parfait — mais pas

l'expérience — la joie de jouer. Cette distinction est centrale dans toute décision d'automatisation.

- La question fondamentale : « Qu'est-ce que cette automatisation libère — et vers quoi ? » Cette question doit précéder toute décision technique.
- Si la réponse à cette question est floue, l'automatisation est probablement prématurée ou mal ciblée.
- Automatiser les tests les plus faciles n'est pas automatiser les tests les plus précieux. La facilité technique et la valeur stratégique ne sont pas synonymes.

Symptômes à observer — Chapitre 1

- L'équipe parle de « taux de couverture » sans jamais parler de « zones à haut risque couvertes ».
- Les initiatives d'automatisation sont systématiquement proposées par les développeurs les plus techniquement curieux — jamais par les PO ou les utilisateurs finaux.
- Personne ne peut répondre clairement à la question : « Une fois ces tests automatisés, que ferons-nous du temps libéré ? »
- L'automatisation est valorisée pour sa sophistication technique plutôt que pour ses effets mesurables sur la qualité ou la confiance.
- Les 80% de cas testés en priorité sont les 80% qui ne cassent jamais.
- L'équipe ressent une certaine fierté à afficher des métriques d'automatisation élevées, même en absence de réduction des bugs en production.

Questions de réflexion — Chapitre 1

1. Quelle est la motivation principale derrière votre dernière initiative d'automatisation ? Était-elle nommée explicitement, ou a-t-elle agi en silence ?

2. Pouvez-vous compléter la phrase : « Nous automatisons [X] pour libérer [qui] de [quoi], afin qu'il/elle puisse [faire quoi] » ? Si non, que révèle cette difficulté ?

3. Y a-t-il des zones de votre SDLC où vous automatisez par habitude ou par confort technique plutôt que par stratégie ?

4. Si vous deviez nommer honnêtement votre motivation principale à automatiser, laquelle des cinq serait-ce ? Êtes-vous à l'aise avec cette réponse ?

5. Qu'est-ce qui dans votre environnement de travail facilite ou décourage les conversations honnêtes sur les motivations réelles d'automatisation ?

Signal de vigilance — Chapitre 1

Danger rouge : L'équipe mesure le succès de ses projets d'automatisation principalement par le nombre de tests créés ou le pourcentage de couverture, sans aucune corrélation avec la réduction effective des incidents en production ou l'amélioration de la confiance dans les releases. Ce découplage entre les métriques d'automatisation et les métriques de qualité réelle est l'un des signaux les plus fiables qu'une automatisation est orientée vers la

Motivation 5 non questionnée plutôt que vers la valeur utilisateur.

CHAPITRE 2

Ce qui doit et ne doit pas être automatisé dans le SDLC réconcilié

Le SDLC réconcilié n'est pas seulement une séquence de phases — c'est un cadre de discernement. Ses six phases et ses trois zones ne définissent pas uniquement ce qu'on fait, mais comment on le fait, et surtout qui le fait — l'humain, la machine, ou les deux ensemble. Ce chapitre explore la cartographie de l'automatisation à travers les zones SENSE, FORGE et CARE : où elle amplifie, où elle nuit, et où elle doit s'arrêter.

2.1 Tout n'est pas automatisable — et tout n'a pas à l'être

L'une des confusions les plus répandues dans l'industrie logicielle est de traiter l'automatisation comme une direction à sens unique : plus on automatise, mieux c'est. Cette conviction, bien qu'intuitivement séduisante, est empiriquement fausse. Il existe des catégories entières d'activités dans le SDLC réconcilié qui résistent à l'automatisation — non pas parce qu'elles sont techniquement difficiles à automatiser, mais parce que leur valeur réside précisément dans l'engagement humain qui les constitue.

Pour comprendre cela, il faut d'abord distinguer trois types d'activités dans tout processus de développement logiciel :

Type d'activité	Caractéristique principale	Posture d'automatisation
Activités déterministes	Même input → même output attendu, toujours	AUTOMATISER sans réserve
Activités contextuel-les	L'output dépend du contexte, du jugement, de l'histoire	ASSISTER — l'humain décide, la machine supporte
Activités relationnelles	La valeur réside dans l'échange humain lui-même	HUMAIN UNIQUEMENT — l'automatisation détruit la valeur

Cette distinction semble simple. En pratique, la frontière entre les trois catégories est souvent floue, et c'est précisément dans cette zone grise que les erreurs d'automatisation les plus coûteuses se produisent. Le SDLC réconcilié, avec ses trois zones SENSE, FORGE et CARE, offre un cadre géographique pour naviguer cette complexité.

2.2 Ce qui doit être automatisé en zone FORGE

La zone FORGE correspond aux phases **IMPLÉMENTER** et **DIFFUSER** du SDLC réconcilié. C'est la zone de la construction et du déploiement — là où le code prend forme, est validé, intégré, et livré. C'est dans cette zone que l'automatisation atteint son plein potentiel, parce que les activités qui s'y trouvent sont majoritairement déterministes.

Les candidats naturels à l'automatisation en zone FORGE incluent :

- **Tests unitaires :** Un test unitaire bien écrit est, par définition, déterministe. Même input, même comportement attendu.
  L'automatisation ici ne dégrade aucune valeur humaine — elle amplifie la rigueur à un niveau qu'aucun humain ne pourrait maintenir manuellement sur la durée.
- **Intégration continue :** Chaque merge déclenche un cycle de vérification standardisé. La règle est connue. Le jugement n'est pas requis à ce stade

— seulement la conformité aux critères définis. L'automatisation est non seulement appropriée, elle est nécessaire à toute vélocité sérieuse.

- **Déploiement continu :** Lorsque les critères de qualité sont satisfaits, le déploiement dans les environnements ciblés peut être entièrement automatisé. Cela élimine les erreurs humaines de procédure, standardise les configurations, et accélère la boucle de feedback.
- **Analyse statique de code :** La vérification automatique du style, des conventions, des vulnérabilités connues et des métriques de complexité est un exemple parfait d'automatisation déterministe. Les règles sont définies par les humains — la vérification est déléguée à la machine.
- **Tests de régression :** Les comportements connus et stables qui ont déjà été testés et validés peuvent être automatisés pour garantir qu'ils ne régressent pas lors de futures modifications. L'automatisation ici libère les équipes QA de la répétition pour qu'elles se concentrent sur ce que nous aborderons en 2.5.
- **Génération de rapports de métriques :** La collecte et l'agrégation des données de qualimétrie — taux d'échec, temps d'exécution, stabilité des builds — sont des activités parfaitement automatisables. Ce qui ne l'est pas, c'est l'interprétation de ces données, qui appartient à la zone CARE.

Le principe directeur en zone FORGE est simple : **si la règle est connue et stable, la vérification peut être automatisée.** Si la règle évolue fréquemment, si elle dépend du contexte, ou si son application requiert du jugement situationnel, la catégorie ASSISTER est plus appropriée.

Une mise en garde importante : automatiser en zone FORGE ne signifie pas confier la définition des règles à la machine. Les seuils de qualimétrie, les critères de réussite, les priorités de test — tout cela reste une responsabilité humaine, exercée en zone SENSE et confirmée en zone CARE. La machine applique les règles. L'humain les définit et les réévalue.

2.3 Ce qui ne doit pas être automatisé en zone SENSE

La zone SENSE correspond aux phases **ÉLICITER** et **DÉFINIR** du SDLC réconcilié. C'est la zone où l'équipe écoute, explore, comprend et formalise. C'est la zone où naissent les spécifications IF…THEN…, où les besoins utilisateurs sont traduits en exigences, où les hypothèses sont questionnées et les ambiguïtés résolues.

Cette zone est fondamentalement humaine. Non pas parce que la technologie ne pourrait pas y jouer un rôle — elle le peut déjà, en tant qu'outil d'assistance — mais parce que la valeur de SENSE réside dans la qualité du jugement exercé, pas dans la vitesse ou la précision d'exécution.

Considérons l'élicitation des besoins. Un outil peut transcrire automatiquement une conversation avec un utilisateur. Il peut en extraire des thèmes, identifier des patterns, suggérer des catégories de besoins. Tout cela est utile — c'est de l'assistance. Mais l'élicitation elle-même — l'art de poser la bonne question au bon moment, de déceler le besoin implicite derrière la demande explicite, de sentir quand l'utilisateur dit « je veux X » mais signifie « j'ai peur de Y » — cette compétence est irréductiblement humaine. Un algorithme peut identifier qu'un utilisateur mentionne souvent le mot « lent » — mais seul un humain peut comprendre si cela signifie que le système est effectivement lent, ou que l'utilisateur se sent incompétent face à une interface mal conçue.

La définition des spécifications est dans la même situation. La structure IF… THEN… peut être assistée par des outils — des gabarits, des vérificateurs de complétude, des détecteurs de contradiction. Mais la décision de ce qu'une spécification doit couvrir, de quel niveau de granularité est approprié, de quelle ambiguïté mérite d'être résolue maintenant plutôt que plus tard — tout cela requiert un jugement humain situé dans le contexte de l'équipe, du produit, et de l'utilisateur.

La priorisation est peut-être l'exemple le plus clair. Des algorithmes sophistiqués peuvent suggérer des priorités basées sur des indicateurs — valeur business

estimée, effort estimé, risque technique, dépendances. Ces suggestions sont précieuses. Mais la décision finale appartient à un humain — généralement le Product Owner — qui intègre des facteurs que l'algorithme ne voit pas : la dynamique politique de l'organisation, les attentes implicites d'un client stratégique, l'état émotionnel de l'équipe, les opportunités de marché à fenêtre temporelle étroite. La priorisation assistée est la bonne posture. La priorisation automatisée est une illusion dangereuse.

La règle en zone SENSE : **la machine éclaire, l'humain décide.**

2.4 Ce qui ne doit pas être automatisé en zone CARE

La zone CARE correspond aux phases **CONFIRMER** et à la grande boucle CONFIRMER → ÉLICITER. C'est la zone où l'équipe évalue, célèbre, apprend, et ajuste. C'est là que se tiennent les cérémonies de qualimétrie, les rétrospectives, les décisions de Go/No-Go, et les post-mortems.

Dans cette zone, l'automatisation peut jouer un rôle d'infrastructure : collecter les données de qualimétrie, agréger les métriques de sprint, générer les rapports de tendance, alerter sur les déviations statistiques. Ces activités sont parfaitement automatisables et libèrent un temps précieux pour ce qui ne l'est pas.

Ce qui ne doit pas être automatisé en zone CARE, c'est l'interprétation et la décision. Prenons la cérémonie de qualimétrie : un tableau de bord automatisé peut présenter les indicateurs — taux de défauts, vélocité, stabilité, satisfaction utilisateur. Mais l'interprétation de ces indicateurs — comprendre pourquoi le taux de défauts a augmenté ce sprint, si c'est parce que la complexité était plus élevée, ou parce qu'un membre de l'équipe traversait une période difficile, ou parce que les spécifications étaient insuffisamment claires — cette interprétation exige une mémoire contextuelle, une empathie, et une connaissance de l'histoire de l'équipe qu'aucun algorithme ne peut posséder.

La décision de Go/No-Go est l'exemple le plus critique. Un système automatisé peut définir des critères binaires — si tous les tests passent et si la couverture dépasse le seuil défini, le déploiement est autorisé. Ce type d'automatisation est utile comme garde-fou de base. Mais une décision de Go/No-Go sur un système critique — un module financier, un système de santé, un composant de sécurité — ne peut pas être entièrement déléguée à un algorithme. Elle requiert l'évaluation de risques qui ne sont pas tous mesurables : la robustesse de l'équipe en cas d'incident post-déploiement, la capacité du support à répondre, les contraintes réglementaires en mouvement, la confiance — ou la méfiance — des utilisateurs finaux.

La règle en zone CARE : **les données sont automatisées, l'interprétation et la décision restent humaines.**

2.5 Le test exploratoire — ce qui ne sera jamais automatisé

Parmi toutes les activités du SDLC réconcilié, le test exploratoire occupe une place unique : c'est la seule qui soit, par définition, non automatisable — non pas pour des raisons techniques, mais pour des raisons fondamentalement conceptuelles.

Le test exploratoire, tel qu'exploré en profondeur dans le Livre 11 de la collection, est un processus de découverte simultanée. Le testeur conçoit et exécute des tests au même moment, guidé par ce qu'il observe, ce qu'il ressent, ce qui l'étonne ou l'inquiète. Il n'y a pas de script préétabli — il y a une intention, une hypothèse, et une réponse adaptative à ce que le système révèle au fur et à mesure.

Un robot ne peut pas explorer. Par définition, un robot exécute ce qu'on lui a dit d'exécuter. Sa trajectoire est définie avant qu'il commence. Même un système d'intelligence artificielle générative qui « explore » un système le fait selon des heuristiques programmées — il suit une logique, même probabiliste. Le test exploratoire humain, lui, suit une curiosité — une capacité à être surpris, à trouver

étrange ce qui semblait normal, à formuler une question que personne n'avait pensé à poser.

C'est cette capacité à être surpris qui est irremplaçable. Les bugs les plus dangereux — ceux qui s'effondrent en production au pire moment, dans des contextes que personne n'avait anticipés — sont presque toujours des bugs que les tests automatisés ne pouvaient pas trouver, parce que personne n'avait pensé à les tester. Ce sont précisément les bugs que le test exploratoire trouve, parce qu'un testeur humain peut improviser, peut suivre une intuition, peut dire « et si j'essayais ça... » sans que personne ne lui ait demandé de le faire.

La règle ToTAL : **le budget de test exploratoire ne doit jamais être sacrifié au profit d'un ratio d'automatisation plus élevé.** Ces deux activités ne sont pas substituables. L'automatisation et le test exploratoire coexistent dans une relation complémentaire irréductible — l'une libère du temps pour l'autre.

2.6 Terrain — La Mission Faucon et le bug invisible à la machine

Cas terrain — Grande entreprise technologique spécialisée en systèmes embarqués temps réel

Cette organisation avait investi des années et des ressources considérables dans la construction d'un pipeline CI/CD de classe mondiale. À l'époque des faits, le pipeline exécutait automatiquement plus de 8 000 tests à chaque intégration, couvrant les composants critiques de leur plateforme embarquée. Les temps d'exécution avaient été optimisés à moins de vingt minutes. L'équipe était fière — à juste titre — de cette infrastructure.

La Mission Faucon était l'un des projets phares de l'organisation : un contrat important avec un client industriel exigeant, impliquant un système de contrôle en temps réel pour des équipements fonctionnant dans des

conditions d'extrême précision. Les enjeux étaient élevés. La pression de livraison l'était aussi.

Trois semaines avant la livraison finale, le pipeline CI/CD est vert. Tous les 8 000 tests passent. Les métriques de qualimétrie sont dans les seuils définis. L'équipe se prépare à annoncer que le système est prêt pour la validation finale par le client.

Un vendredi après-midi, une testeuse — appelons-la Élise — décide de passer quelques heures à explorer le système dans un contexte non planifié. Elle n'a pas de script. Elle suit une intuition : le comportement du système dans des séquences d'initialisation non standard l'a toujours légèrement préoccupée, sans qu'elle puisse le formaliser. Elle explore. Elle improvise. Elle teste des séquences que personne n'avait jamais définies dans les cas officiels.

Quarante minutes plus tard, elle reproduit un bug critique. Dans un scénario précis de réinitialisation à chaud — possible en conditions réelles d'opération sur le terrain — le système entre dans un état de latence incohérent qui dépasse les tolérances du contrat. Le bug est déterministe une fois qu'on sait comment le reproduire. Mais pour le produire, il fallait penser à le chercher.

Aucun des 8 000 tests automatisés ne couvrait cette séquence. Non pas parce que le pipeline était mal conçu — il était excellent. Mais parce que personne n'avait spécifié ce scénario, et que les tests automatisés ne testent que ce qu'on a spécifié.

Le bug a été corrigé avant la livraison. La Mission Faucon a été livrée avec succès. Mais sans Élise et sa session exploratoire non planifiée un vendredi après-midi, il ne l'aurait pas été.

Ce que cette anecdote enseigne : Un pipeline d'automatisation exemplaire ne remplace pas le test exploratoire. Il le rend possible, en libérant du temps pour l'exploration. Les deux ne sont pas en compétition — ils sont complémentaires et irremplaçables l'un de l'autre.

2.7 Scène DREAM-TEAM — La carte d'automatisation d'ORIGINALIUM

Scène DREAM-TEAM — ORIGINALIUM · Atelier de cartographie d'automatisation

Rania a proposé un atelier d'une demi-journée. L'objectif : cartographier toutes les activités du SDLC d'ORIGINALIUM sur une grille à trois colonnes — AUTOMATISER / ASSISTER / HUMAIN UNIQUEMENT. Pas de jugement. Pas de défense de territoire. Juste une lecture honnête de l'état actuel et de l'état souhaitable.

L'équipe commence par les activités de la zone FORGE. L'accord est rapide : les tests unitaires, l'intégration continue, les déploiements en environnement de développement et de staging, les analyses statiques — tout ça dans la colonne AUTOMATISER. Unanime. Confortable.

On passe à la zone SENSE. C'est là que ça se complique. Karim propose de mettre la « génération des cas de test » dans la colonne AUTOMATISER. Théo fronce les sourcils.

> **Théo :** *La génération, ou la suggestion ? Parce que je pense que ce sont deux choses très différentes.*
>
> **Karim :** *La suggestion, clairement. Un outil peut suggérer des cas basés sur les spécifications. Mais quelqu'un doit valider, enrichir, prioriser.*
>
> **Rania :** *Alors c'est ASSISTER. L'outil suggère. L'humain décide et enrichit. On ne met rien dans AUTOMATISER si un humain doit*

> *encore valider.*

La distinction ASSISTER vs AUTOMATISER devient la plus riche de l'atelier. L'équipe commence à réaliser que beaucoup de leurs activités qu'ils pensaient automatisées étaient en fait *assistées* — un humain vérifiait encore, même brièvement, même inconsciemment.

À la fin de l'atelier, le résultat est révélateur :

Zone

AUTOMATISER

ASSISTER

HUMAIN UNIQUEMENT

SENSE

8%

52%

40%

FORGE

65%

28%

7%

CARE

22%

55%

23%

Global

~30%

~46%

~24%

> **Rania :** *46% dans ASSISTER. C'est la majorité de notre SDLC. Et c'est la zone qu'on investit le moins. On a des outils d'automatisation pour FORGE, et des processus humains pour SENSE et CARE. Mais pour la zone ASSISTER, qui est partout ?*
>
> **Théo :** *On improvise. Et improviser dans 46% de son SDLC, ce n'est pas une stratégie.*
>
> **Karim :** *Donc notre prochain chantier n'est pas d'automatiser plus. C'est de mieux définir comment on assiste.*

C'est l'une des conclusions les plus importantes qu'ORIGINALIUM tirera de toute sa réflexion sur l'automatisation : la zone ASSISTER est la plus importante et la moins bien adressée dans presque toutes les équipes. Non pas parce qu'elle est négligée par mauvaise volonté, mais parce qu'elle est trop subtile pour être entièrement automatisée et trop répétitive pour être entièrement humaine. C'est précisément là que la réflexion doit se concentrer.

À retenir — Chapitre 2

- Toutes les activités du SDLC réconcilié appartiennent à trois catégories : AUTOMATISER (déterministe), ASSISTER (contextuel), ou HUMAIN UNIQUEMENT (relationnel).
- La zone FORGE est le territoire naturel de l'automatisation : tests unitaires, CI, CD, analyse statique, tests de régression. L'humain définit les règles, la machine les applique.
- La zone SENSE — ÉLICITER et DÉFINIR — est fondamentalement humaine. L'assistance est bienvenue. L'automatisation des décisions est une illusion dangereuse.
- La zone CARE délègue la collecte de données à la machine, mais garde l'interprétation et la décision pour l'humain. Un Go/No-Go ne peut pas être entièrement algorithmique sur des systèmes critiques.
- Le test exploratoire est par définition non automatisable. Ce n'est pas une faiblesse — c'est sa valeur même. Un testeur humain peut être surpris. Un robot, non.
- La zone ASSISTER représente environ 46% du SDLC réconcilié et est la moins bien adressée par la majorité des équipes. C'est là que l'investissement stratégique est le plus précieux.

Symptômes à observer — Chapitre 2

- L'équipe n'a pas de budget de temps explicitement protégé pour le test exploratoire — il est constamment sacrifié au profit de l'exécution de la régression automatisée.
- Les décisions de priorisation sont systématiquement déléguées à des

algorithmes ou des outils de scoring sans validation humaine explicite.

- Les séances d'élicitation ou de définition des spécifications sont raccourcies au motif que « l'IA peut compléter ».
- Personne dans l'équipe ne peut répondre à la question : « Quelle proportion de notre SDLC est dans la zone ASSISTER, et comment la gérons-nous ? »
- Les post-mortems sont remplacés par des rapports automatisés d'analyse des logs — sans conversation humaine, sans réflexion sur les causes profondes.

Questions de réflexion — Chapitre 2

1. Si vous deviez cartographier votre propre SDLC sur la grille AUTOMATISER / ASSISTER / HUMAIN UNIQUEMENT, quelle serait la répartition approximative ? Qu'est-ce que ce résultat vous révèle ?
2. Quelle proportion du temps de votre équipe QA est consacrée au test exploratoire ? Ce ratio est-il intentionnel ou accidentel ?
3. Y a-t-il dans votre SDLC des décisions importantes qui ont été progressivement déléguées à des algorithmes sans que cette délégation ait été explicitement décidée ?
4. Comment votre équipe définit-elle la frontière entre « assister » et « automatiser » dans la pratique quotidienne ?
5. La Mission Faucon de votre équipe — quel bug critique n'a jamais été trouvé par votre pipeline automatisé ? Qu'est-ce que cela vous dit sur vos zones d'ombre ?

Signal de vigilance — Chapitre 2

Danger rouge : L'équipe n'a aucune session de test exploratoire planifiée dans son calendrier de sprint — ni pour les nouvelles fonctionnalités, ni pour les zones à haut risque. Si le test exploratoire n'existe que « quand on a le temps », il n'existe pas. Un pipeline d'automatisation sans test exploratoire complémentaire est un système qui sait parfaitement exécuter les tests qu'il a appris à faire, et qui est aveugle à tout le reste. C'est la définition de la fausse sécurité.

CHAPITRE 3

Les quatre pièges de l'automatisation mal ciblée

Connaître les motivations de l'automatisation et comprendre ce qui devrait ou non être automatisé dans le SDLC réconcilié est nécessaire — mais pas suffisant. Il faut également connaître les pièges dans lesquels même les équipes les plus intentionnelles tombent. Ces quatre pièges sont indépendants des technologies utilisées, des secteurs d'activité, et des niveaux de maturité des équipes. Ils se manifestent partout, sous des formes différentes, avec la même régularité décourageante.

3.1 Piège 1 — Automatiser le mauvais problème

Piège 1 / 4

Automatiser le mauvais problème

"Si votre processus est mauvais, l'automatiser le rendra plus rapidement mauvais."

Cette formulation est souvent attribuée à diverses sources dans la littérature sur la gestion de processus, mais son origine importe peu — sa vérité est universelle et vérifiable dans pratiquement n'importe quelle organisation ayant traversé une initiative d'automatisation ambitieuse.

Le piège est séduisant précisément parce qu'il se déguise en solution. Une équipe fait face à un processus lent, douloureux, source de friction quotidienne. La réponse instinctive est d'automatiser ce processus pour qu'il soit moins douloureux. Mais si le processus est mal conçu à la base — s'il contient des étapes inutiles, des redondances, des dépendances circulaires ou des décisions mal positionnées —, l'automatisation ne résout pas le problème. Elle le cimente.

Prenons un exemple courant dans l'industrie logicielle. Une grande firme conseil en TI gère ses demandes de déploiement via un processus d'approbation à huit étapes impliquant quatre départements différents. Le processus est lent, les erreurs de communication sont fréquentes, les délais sont imprévisibles. La direction décide d'automatiser le processus de routing des demandes pour accélérer les approbations.

Trois mois plus tard, le routing automatisé fonctionne parfaitement. Les demandes sont acheminées instantanément vers les bons approbateurs. Mais le temps total d'approbation n'a pas significativement diminué. Pourquoi ? Parce que le bottleneck n'était pas le routing — c'était la pertinence des huit étapes d'approbation elles-mêmes. Deux de ces étapes étaient redondantes. Une troisième n'avait de sens que pour des déploiements sur des systèmes legacy qui ne représentaient plus que 3% du volume total. L'automatisation a rendu le

processus plus rapide à chaque étape — mais n'a pas questionné si toutes les étapes devaient exister.

La règle ToTAL face au Piège 1 est stricte : **avant d'automatiser un processus, simplifier le processus.** Identifier les étapes qui existent par inertie plutôt que par nécessité. Éliminer les redondances. Clarifier les décisions mal positionnées. Ce travail de simplification est moins glamour que l'automatisation — il ne produit pas de dashboards impressionnants — mais c'est lui qui génère le vrai gain de valeur.

La question à se poser avant chaque initiative d'automatisation de processus : *« Si nous devions concevoir ce processus à partir de zéro aujourd'hui, ressemblerait-il à ce que nous allons automatiser ? »* Si la réponse est non, le travail de simplification précède l'automatisation.

Dans le domaine du test logiciel, ce piège se manifeste souvent sous la forme suivante : une équipe automatise ses cas de test existants — ceux qui ont été écrits il y a deux ans, dans un contexte différent, pour une architecture différente. Ces cas de test ont peut-être été partiellement mis à jour, mais leur structure fondamentale reflète des hypothèses obsolètes sur le comportement du système. Automatiser des cas de test périmés, c'est encapsuler dans du code les erreurs de conception du passé et les exécuter à chaque build avec une efficacité parfaite.

3.2 Piège 2 — La dette d'automatisation

Piège 2 / 4

La dette d'automatisation

La dette d'automatisation est souvent plus dangereuse que la dette technique, parce qu'elle est invisible.

La dette technique est un concept bien établi dans l'industrie : le coût différé des décisions de développement qui sacrifient la qualité à long terme pour la vitesse à court terme. La dette d'automatisation est analogue — mais elle est moins bien comprise, moins souvent mesurée, et donc plus facilement laissée s'accumuler.

Un script de test automatisé qui a été écrit il y a dix-huit mois et n'a pas été révisé depuis est une forme de dette d'automatisation. Si le système qu'il teste a changé, le script teste peut-être quelque chose qui n'existe plus — ou, pire, il passe sur un comportement nouveau parce que ses assertions n'ont pas été mises à jour pour capturer ce comportement. Un script qui passe sans erreur sur un système modifié, non pas parce que le système fonctionne correctement, mais parce que le script ne sait pas quoi vérifier, est plus dangereux qu'un script qui échoue.

Un pipeline CI/CD qui a été construit par une seule personne — souvent avec une excellente compréhension du système à l'époque — et qui n'a jamais été documenté ni partagé est une dette d'automatisation. Le jour où cette personne quitte l'organisation, le pipeline devient une boîte noire que personne n'ose modifier, parce que personne ne comprend vraiment ce qu'il fait à l'intérieur. Ces pipelines-boîtes-noires sont remarquablement fréquents — et ils créent une dépendance organisationnelle à une personne unique que toute bonne architecture d'automatisation devrait chercher à éliminer.

La dette d'automatisation est invisible de trois façons :

- **Elle ne signale pas son existence.** Un script obsolète qui passe toujours ne produit pas de message d'erreur. Il continue de donner une impression de sécurité même si cette sécurité est devenue fictive.
- **Elle n'est pas mesurée.** La plupart des équipes mesurent leur taux de couverture de test automatisé — très peu mesurent le taux de « tests pertinents » dans leur portfolio automatisé. Ces deux métriques peuvent diverger radicalement avec le temps.

- **Elle s'accumule silencieusement.** Chaque sprint, on ajoute des tests automatisés sans en retirer. Deux ans plus tard, le portfolio contient des centaines de tests écrits pour des fonctionnalités modifiées ou supprimées. Le volume impressionne — la valeur, elle, a diminué.

La règle ToTAL face au Piège 2 : **chaque test automatisé ajouté doit être accompagné d'une politique explicite de révision et de dépréciation.** Un test automatisé qui n'est plus pertinent doit être désactivé ou supprimé — non pas archivé, non pas commenté, supprimé. Un portfolio de tests automatisés, comme un jardin, doit être régulièrement entretenu pour rester utile.

La proposition concrète du SDLC réconcilié : intégrer une « session de désherbage » dans chaque cycle trimestriel — une revue systématique du portfolio de tests automatisés visant à identifier les scripts obsolètes, les assertions qui ne capturent plus les comportements attendus, et les pipelines dont la complexité dépasse la compréhension collective de l'équipe.

3.3 Piège 3 — L'atrophie des compétences

Piège 3 / 4

L'atrophie des compétences

On n'automatise jamais entièrement une compétence qu'on veut maintenir.

L'atrophie des compétences est peut-être le piège le moins visible et le plus durable de tous. C'est le phénomène par lequel une équipe perd progressivement la capacité de faire manuellement ce qu'elle a entièrement automatisé — jusqu'au jour où l'automatisation défaille, et où personne ne sait plus comment reprendre la main.

L'exemple le plus cité dans la littérature de sécurité de l'aviation est celui des pilotes de ligne. Les avions modernes sont capables d'effectuer des vols entiers

en pilote automatique, de la montée initiale à l'approche finale. Cette automatisation est précieuse — elle réduit la fatigue du pilote, améliore la précision des trajectoires, et libère la capacité cognitive du cockpit pour la gestion des situations exceptionnelles. Mais les agences de sécurité aérienne dans plusieurs pays ont observé avec inquiétude que certains pilotes très expérimentés perdaient progressivement leur aisance dans la gestion manuelle d'un aéronef — précisément parce qu'ils volaient presque exclusivement en mode automatique. Or, les situations où le pilotage manuel est requis sont exactement celles où un incident majeur est en cours — les moments où la compétence doit être la plus affûtée.

La même dynamique existe dans l'industrie logicielle, sous des formes peut-être moins dramatiques mais tout aussi réelles. Une équipe qui automatise entièrement son processus de déploiement perd progressivement la connaissance pratique des étapes manuelles de ce déploiement. Un jour, le pipeline tombe en panne dans un contexte critique — une release d'urgence pour corriger un bug en production. Personne ne se souvient exactement comment déployer manuellement. Le playbook existe quelque part, mais il n'a pas été mis à jour depuis deux ans. La situation d'urgence devient une crise d'urgence.

Une équipe QA qui automatise 100% de ses tests de régression perd progressivement la compétence de concevoir des tests à la main. Les membres de l'équipe deviennent des opérateurs d'un système de test plutôt que des concepteurs de stratégie de test. La distinction est fondamentale : un opérateur gère un système qui existe — un concepteur crée un système qui correspond au risque réel. Si le framework de test automatisé change de version majeure, ou est abandonné par son éditeur, l'équipe d'opérateurs est démunie face à la migration.

La règle ToTAL face au Piège 3 est l'une des plus importantes de la collection : **on n'automatise jamais entièrement une compétence qu'on veut maintenir.** Cela signifie concrètement :

- Maintenir des exercices périodiques de déploiement manuel — au moins trimestriellement — même lorsque le déploiement automatisé fonctionne parfaitement.
- Inclure dans le calendrier de sprint des sessions de conception de tests manuels, même lorsque le portfolio automatisé est robuste.
- Documenter les procédures manuelles et les maintenir à jour en parallèle des automatisations correspondantes.
- Former les nouveaux membres de l'équipe en commençant par les processus manuels avant de leur enseigner leur automatisation.

Ce n'est pas de la nostalgie pour les méthodes artisanales. C'est de la résilience organisationnelle. Une équipe qui sait faire manuellement ce qu'elle fait automatiquement est une équipe qui comprend réellement ce qu'elle fait. Et cette compréhension est irremplaçable.

3.4 Piège 4 — La fausse sécurité

Piège 4 / 4

La fausse sécurité

100% de tests qui passent ≠ 100% de qualité.

Le Piège 4 est peut-être le plus insidieux de tous, parce qu'il se manifeste précisément dans les moments où l'équipe se sent la plus confiante. Un pipeline entièrement vert. Tous les tests qui passent. La qualimétrie dans le vert. Les dashboards apaisants. Et pourtant — le bug critique qui attend dans l'ombre, dans un angle mort que l'automatisation ne voit pas.

La fausse sécurité naît d'une équation fondamentalement incorrecte : *confondre l'absence de signal d'erreur avec la présence de qualité.* Un test automatisé qui passe signifie exactement une seule chose : le comportement testé correspond

aux assertions définies dans ce test. Il ne dit rien sur les comportements non testés — et ce sont précisément ces comportements qui contiennent les bugs les plus dangereux.

Cette équation incorrecte est renforcée par plusieurs facteurs organisationnels :

La pression de livraison. Quand le pipeline est vert et que la date de release approche, la tentation est forte d'interpréter « tous les tests passent » comme « le système est prêt ». Cette interprétation est compréhensible sous pression — mais elle est fausse. « Tous les tests passent » signifie seulement « tous les comportements que nous avons choisi de tester répondent à nos attentes ». C'est une déclaration sur notre capacité à tester, pas sur la qualité du système.

Les métriques de couverture. Un taux de couverture de code de 85% semble impressionnant — et il l'est, en termes d'effort investi. Mais la couverture de code mesure la proportion de lignes de code exécutées par les tests, pas la proportion de comportements critiques validés. On peut avoir 100% de couverture de code et des bugs catastrophiques dans les cas limites, les interactions entre modules, ou les comportements sous charge.

L'habitude du vert. Quand un pipeline est vert depuis longtemps, l'équipe développe une confiance conditionnée. Elle cesse de questionner les résultats et commence à les consommer comme des certitudes. Cette confiance conditionnée est particulièrement dangereuse lors des grandes refactorisations ou des migrations d'architecture, où le comportement peut changer de manière subtile sans que les tests existants le détectent.

La règle ToTAL face au Piège 4 : **traiter les résultats d'automatisation comme un signal, pas comme une conclusion.** Le pipeline vert est une condition nécessaire à la release — jamais une condition suffisante. La conclusion sur la qualité appartient à une interprétation humaine qui inclut les résultats d'automatisation, les résultats du test exploratoire, les feedbacks des utilisateurs, et le jugement professionnel de l'équipe QA.

3.5 Terrain — Le studio de développement de jeux vidéo et le DLC catastrophique

Cas terrain — Studio de développement de jeux vidéo

Ce studio avait investi massivement dans son infrastructure d'automatisation pendant les deux années précédant les faits. À l'époque du DLC en question, l'équipe disposait d'un portfolio de 12 400 tests automatisés couvrant les systèmes de combat, de navigation, de sauvegarde, d'interface, et d'économie du jeu. Le pipeline de déploiement continu était l'un des plus sophistiqués du secteur — des collègues d'autres studios venaient parfois l'observer comme modèle de maturité DevOps.

Le DLC — un contenu additionnel majeur introduisant une nouvelle région, de nouveaux mécanismes de gameplay, et une extension de la progression narrative — était prêt pour la release. Tous les 12 400 tests étaient verts. La couverture de code était à 79%. L'équipe était confiante. La date de release était maintenue. La campagne marketing était en cours.

Le DLC est lancé. Dans les premières heures, les forums des joueurs s'enflamment. Un bug critique : dans certaines conditions de transition entre la région principale et la nouvelle région, l'inventaire du personnage se corrompait partiellement, entraînant la perte de certains objets rares. Les objets rares représentent des heures, parfois des dizaines d'heures, d'investissement des joueurs. Les plaintes sont massives. L'humeur est grave.

L'équipe analyse. Le bug est reproductible — mais seulement dans un scénario très précis : une transition effectuée pendant que le personnage portait simultanément deux objets de la même catégorie ayant des buffs conflictuels, et uniquement si la transition survenait dans une fenêtre temporelle de moins de deux secondes après l'ouverture d'un menu spécifique. Un scénario absurde, improbable — et pourtant, avec des millions

de joueurs, statistiquement inévitable.

Aucun des 12 400 tests automatisés ne couvrait cette interaction. Non pas parce que personne n'était compétent — c'était une excellente équipe. Mais parce que personne n'avait pensé à spécifier ce cas. Et les tests automatisés ne testent que ce qu'on a spécifié.

Un testeur exploratoire chevronné aurait peut-être trouvé ce bug — non pas en le cherchant spécifiquement, mais en suivant le genre d'intuition qui pousse à « essayer des choses bizarres » dans les systèmes d'inventaire pendant les transitions de zone. Mais l'équipe n'avait pas réservé de budget de test exploratoire pour le DLC — la confiance dans le pipeline automatisé avait rendu ce budget difficile à justifier.

Ce que cette anecdote enseigne : 12 400 tests verts et une couverture à 79% ne protègent pas contre les bugs dans les interactions non spécifiées. Le studio a appris deux leçons durables : réserver systématiquement du temps de test exploratoire pour chaque release majeure, et ne jamais interpréter le pipeline vert comme une déclaration de qualité — seulement comme un signal de conformité aux critères définis.

À retenir — Chapitre 3

- **Piège 1 :** Automatiser un processus mauvais le rend plus rapidement mauvais. Simplifier avant d'automatiser. La question clé : « Si on concevait ce processus à partir de zéro aujourd'hui, ressemblerait-il à ce qu'on s'apprête à automatiser ? »
- **Piège 2 :** La dette d'automatisation est invisible et s'accumule silencieusement. Les tests obsolètes, les scripts non maintenus, et les

pipelines-boîtes-noires sont des dettes aussi réelles que la dette technique — mais plus difficiles à mesurer.

- **Piège 3 :** L'atrophie des compétences est la conséquence directe de l'automatisation totale. On n'automatise jamais entièrement une compétence qu'on veut maintenir. Des exercices manuels périodiques sont une forme de résilience organisationnelle, pas de nostalgie.
- **Piège 4 :** La fausse sécurité est le piège le plus insidieux. Un pipeline vert est une condition nécessaire, jamais suffisante. Les tests automatisés testent ce qu'on a choisi de tester — pas ce qui pourrait potentiellement échouer.
- Les quatre pièges sont indépendants et peuvent coexister dans la même équipe. Un diagnostic honnête nécessite de les chercher simultanément.

Symptômes à observer — Chapitre 3

- L'équipe interprète systématiquement « tous les tests passent » comme « le système est prêt » — sans autre forme de validation.
- Le portfolio de tests automatisés n'a jamais fait l'objet d'une révision de pertinence. On n'y ajoute, on n'y enlève jamais rien.
- Il n'existe pas de procédure documentée à jour pour les processus clés si les outils d'automatisation devenaient indisponibles.
- Les membres de l'équipe seraient incapables de déployer manuellement sans le pipeline CI/CD.
- L'équipe automatise de nouveaux tests sans jamais analyser si le

processus sous-jacent mérite d'être simplifié d'abord.

- Les bugs en production proviennent systématiquement d'interactions entre composants, jamais de comportements couverts par les tests automatisés.

Questions de réflexion — Chapitre 3

1. Votre équipe a-t-elle déjà réalisé un audit de pertinence de son portfolio de tests automatisés ? Si oui, que vous a-t-il appris ? Si non, pourquoi ?
2. Quel est votre plan de continuité si votre principal outil d'automatisation devient indisponible pour 48 heures ? Avez-vous testé ce plan ?
3. Quand avez-vous pour la dernière fois exécuté un processus critique manuellement, délibérément, pour maintenir la compétence ? Qu'avez-vous redécouvert ?
4. Y a-t-il des processus que votre équipe automatise en ce moment qui mériteraient d'être simplifiés d'abord ? Qu'est-ce qui empêche cette simplification ?
5. Comment votre équipe distingue-t-elle « le système est qualifié » de « nos tests sont verts » ? Existe-t-il des critères humains explicites au-delà du pipeline ?

Signal de vigilance — Chapitre 3

Danger rouge — Combinaison des quatre pièges : Si votre équipe automatise sans simplifier les processus d'abord (Piège 1), n'a jamais révisé son portfolio de tests pour en retirer les scripts obsolètes (Piège 2), ne maintient pas de compétences manuelles en parallèle (Piège 3), et interprète le pipeline vert comme une déclaration de qualité suffisante (Piège 4) — votre système d'automatisation est en réalité un système de génération de fausse confiance. Il ne protège pas le produit — il protège l'équipe du sentiment d'inconfort. Ces deux fonctions ne sont pas équivalentes.

CHAPITRE 4

ONETOOL : la tentation de l'outil miracle

Chaque génération technologique produit son ONETOOL — l'outil qui promet d'automatiser le pipeline complet, de supprimer la complexité, de rendre les décisions techniques triviales. L'histoire de l'industrie logicielle est en partie l'histoire de ces promesses et de leur confrontation avec la réalité. Ce chapitre n'est pas un réquisitoire contre les outils d'automatisation — il en existe d'excellents. C'est une réflexion sur la relation que les équipes entretiennent avec ces outils, et sur ce que cette relation peut révéler de sain ou de problématique dans leur culture d'automatisation.

4.1 L'histoire de ONETOOL

ONETOOL n'est pas le nom d'un outil réel. C'est un archétype — le nom qu'on donne dans la collection ToTAL à tout outil qui se présente comme la solution complète à un problème complexe. Au fil des décennies, ONETOOL a pris des

noms différents selon les ères technologiques, mais son profil de promesses est étrangement constant :

- *« Un seul outil pour tout votre pipeline »*
- *« Zéro configuration, résultats immédiats »*
- *« Remplace vos outils actuels et vos processus manuels »*
- *« Intégration native avec tout ce que vous utilisez déjà »*
- *« Vos équipes adorent l'utiliser »*

Chacune de ces promesses peut être vraie. Ce n'est pas le problème. Le problème, c'est ce que ces promesses induisent comme comportement dans les équipes qui les reçoivent : la suspension du discernement critique.

L'histoire type d'un ONETOOL se déroule toujours à peu près de la même manière. Un mois un : démonstration. L'outil est impressionnant. Les cas d'usage présentés correspondent parfaitement aux problèmes de l'équipe. L'achat est justifié, parfois avec enthousiasme, parfois sous la pression de la direction qui a vu la même démonstration à une conférence.

Mois deux à quatre : euphorie. L'outil tient ses promesses initiales. L'équipe migre ses processus existants dans ONETOOL. La productivité semble augmenter. Les dashboards sont jolis. Les métriques ressemblent à celles de la démo.

Mois cinq à huit : friction. Les cas d'usage non couverts par la démonstration commencent à émerger. Les intégrations promises sont partielles ou instables. La configuration qui semblait simple révèle des couches de complexité croissante. Un ou deux membres de l'équipe deviennent les « experts ONETOOL » — les seuls qui comprennent vraiment comment faire fonctionner certaines parties du système.

Mois neuf à douze : dépendance. L'équipe a tellement migré dans ONETOOL qu'un retour en arrière semble impossible. Les anciens processus ont été désactivés. Les compétences pour les faire fonctionner manuellement ont commencé à s'atrophier. ONETOOL n'est plus un outil — c'est une infrastructure critique. Et

comme toute infrastructure critique, toute tentative de la questionner rencontre une résistance institutionnelle.

Mois treize et au-delà : paralysie. Un changement de version majeur de ONETOOL rompt deux workflows critiques. Ou le fournisseur est racheté et annonce la fin du support. Ou les frais de licence augmentent de 40% et l'équipe n'a plus de levier de négociation parce qu'elle ne peut pas partir. C'est à ce moment que le coût réel de la dépendance devient visible — toujours trop tard.

Cette trajectoire n'est pas universelle. Certains ONETOOL sont d'excellents outils qui créent de la valeur durable pour les organisations qui les adoptent avec discernement. La différence entre le succès et la paralysie se joue précisément dans ce discernement — exercé avant l'adoption, maintenu tout au long du cycle de vie, et jamais suspendu parce que « ça fonctionne pour l'instant ».

4.2 La loi de la dépendance à l'outil

La collection ToTAL propose une formulation que nous appelons la Loi de la dépendance à l'outil :

> *« Plus un outil est puissant, plus la dépendance à cet outil est difficile à reconnaître — et plus le coût de sortie est élevé. »*

Cette loi mérite d'être développée sur chacun de ses termes.

« Plus un outil est puissant » — La puissance d'un outil se mesure à la proportion de valeur qu'il génère. Plus il génère de valeur, plus l'équipe réorganise ses processus autour de lui. Cette réorganisation est rationnelle et souvent bénéfique — c'est la nature même de l'adoption d'un bon outil.

« plus la dépendance est difficile à reconnaître » — Quand un outil fonctionne bien, la dépendance est invisible. On ne perçoit pas qu'on dépend de l'air qu'on respire — jusqu'au moment où il manque. De même, une équipe qui

utilise ONETOOL depuis deux ans pour orchestrer son pipeline complet ne perçoit pas la dépendance — elle perçoit simplement « notre façon de travailler ». C'est au moment où ONETOOL est indisponible, ou doit être remplacé, que la dépendance devient visible.

« plus le coût de sortie est élevé » — Le coût de sortie d'un outil inclut trois dimensions souvent sous-estimées : le coût de migration technique (reconstruire les workflows dans un autre outil), le coût de compétence (former l'équipe à un nouveau paradigme), et le coût de données (migrer les historiques, les configurations, les artefacts produits par l'outil). Ce coût triple est souvent l'argument le plus puissant pour rester dans ONETOOL même lorsqu'on reconnaît ses limites.

La règle ToTAL face à la dépendance : **trois principes de relation saine avec tout outil d'automatisation.**

- **Comprendre ce que l'outil fait.** Pas superficiellement — structurellement. L'équipe devrait être capable d'expliquer ce que l'outil fait à l'intérieur de ses boîtes noires, même approximativement. Un outil qu'on ne comprend pas est un outil qu'on ne contrôle pas.
- **Maintenir la capacité à faire sans lui.** Ce n'est pas un appel à doubler chaque automatisation d'un équivalent manuel. C'est l'assurance que les processus critiques peuvent être exécutés, dégradés si nécessaire, sans cet outil spécifique. Cette capacité doit être testée périodiquement — pas théorisée.
- **Avoir un plan de sortie viable.** Avant d'adopter tout outil d'automatisation significatif, l'équipe devrait être capable de répondre : « Si nous devions sortir de cet outil dans six mois, comment ferions-nous ? » Si la réponse est « on ne pourrait pas », l'adoption mérite d'être reconsidérée ou structurée différemment.

4.3 Comment évaluer un outil d'automatisation

Le SDLC réconcilié propose une grille d'évaluation des outils d'automatisation structurée en cinq dimensions. Cette grille n'est pas destinée à disqualifier les outils ambitieux — elle est destinée à informer l'adoption et à structurer la conversation entre l'équipe technique, le Product Owner, et la direction.

Dimension	Question clé	Signal de danger
Périmètre réel	Qu'automatise-t-il exactement ? Qu'est-ce qu'il ne fait PAS que la démonstration laisse entendre ?	La démonstration ne couvre que les cas les plus simples. Les cas limites sont « configurables ».
Atrophie induite	Quelles compétences cette automatisation rend-elle progressivement inutiles ? Voulons-nous les maintenir ?	L'outil automatise des compétences qu'on ne peut pas se permettre de perdre entièrement.
Coût de sortie	Si on devait quitter cet outil dans 12 mois, quel serait le coût réel ? Technique, humain, financier ?	Le vendeur ne peut pas répondre à cette question — ou détourne la conversation.
Transparence interne	Comprend-on ce que l'outil fait à l'intérieur ? Peut-on expliquer son comportement quand il échoue ?	L'outil est une boîte noire complète. Les logs d'erreur sont cryptiques. Le support est le seul recours.
Dépendance organisationnelle	Combien de personnes dans l'équipe comprennent l'outil suffisamment pour le maintenir ? Est-ce suffisant ?	Une seule personne maîtrise l'outil. Son départ créerait une crise.

Cette grille doit être utilisée avant l'adoption d'un outil, mais aussi lors de revues périodiques des outils déjà en place. Des outils adoptés avec toutes les garanties peuvent évoluer dans une direction moins favorable — changements de propriété,

pivots de stratégie produit, augmentations de tarif, abandon de fonctionnalités clés. La relation avec un outil d'automatisation est un contrat dynamique qui mérite une réévaluation régulière.

Un dernier point sur l'évaluation : **le meilleur moment pour poser ces questions est avant l'adoption, pas après.** Une fois l'outil intégré dans le pipeline de production, une fois les workflows restructurés autour de lui, une fois les compétences alternatives partiellement atrophiées — répondre honnêtement à ces questions devient politiquement difficile. La résistance au changement n'est pas de la mauvaise volonté — c'est une réponse rationnelle au coût de sortie réel. C'est précisément pourquoi le discernement doit s'exercer avant, lorsque le coût de sortie est encore nul.

4.4 Scène DREAM-TEAM — La tentation ONETOOL

Scène DREAM-TEAM — ORIGINALIUM · Évaluation d'un outil d'automatisation

Un representant commercial d'un fournisseur d'outils d'automatisation vient présenter sa solution à l'équipe d'ORIGINALIUM. La démonstration est impressionnante : un pipeline complet — de l'élicitation des exigences au déploiement en production — orchestré depuis une seule interface. Les métriques temps réel. L'intégration avec les outils existants en quelques clics. Le tableau de bord de qualimétrie qui se met à jour automatiquement. La salle est silencieuse d'une façon qui indique que tout le monde est séduit.

Le representant ouvre la session de questions.

> **Rania :** *Quand vous dites « intégration native avec nos outils existants », ça signifie quoi exactement ? Nos outils actuels de*

> *gestion des exigences continueront à fonctionner indépendamment ?*
>
> **Représentant :** *Absolument. La migration est transparente. Vous pouvez même fonctionner en parallèle pendant la transition.*
>
> **Théo :** *Combien de temps dure « la transition », en pratique, pour une équipe de notre taille ?*
>
> **Représentant :** *Nos clients similaires complètent généralement en quatre à six mois.*
>
> **Théo :** *Et pendant ces quatre à six mois, on maintient deux systèmes en parallèle ?*
>
> **Représentant :** *Oui, c'est la pratique recommandée pour une migration sans disruption.*

Puis Karim pose la question. Simplement. Calmement. Comme si la réponse l'intéressait davantage que la démonstration.

> **Karim :** *Si votre outil est indisponible pendant une semaine — une panne, une mise à jour forcée, une défaillance d'infrastructure — est-ce que notre équipe peut continuer à travailler ? Et si oui, comment ?*

Le representant hésite. Pas longtemps — deux secondes, peut-être trois. Mais l'hésitation est là.

> **Représentant :** *Nos SLAs garantissent 99,9% de disponibilité. Les temps d'arrêt sont extrêmement rares et—*
>
> **Karim :** *Ce n'est pas ma question. Ma question est : si ça arrive, on peut continuer à travailler ? Oui ou non, et comment ?*

Nouveau silence. Le representant consulte ses notes.

> **Représentant :** *Il existe des modes dégradés pour certaines fonctionnalités. Notre équipe de support peut—*
>
> **Karim :** *C'est correct. Merci.*

Après le départ du representant, l'équipe reste quelques minutes dans la salle.

> **Karim :** *La réponse à ma question me dit tout ce que j'ai besoin de savoir. Si on ne peut pas continuer à travailler sans cet outil pendant une semaine, ce n'est pas un outil — c'est une infrastructure critique à risque unique. Et on ne décide pas d'adopter une infrastructure critique à risque unique sur la base d'une démonstration de quarante-cinq minutes.*
>
> **Rania :** *L'outil n'est peut-être pas mauvais. Mais on a besoin de réponses à cette question avant d'aller plus loin.*

> **Théo :** *Exactement. On demande une proof of concept avec nos vrais cas, pas leur meilleur cas. Et on leur demande de documenter le mode dégradé avant de signer quoi que ce soit.*

ORIGINALIUM demandera ces éléments. Certains seront fournis de manière satisfaisante. D'autres ne le seront pas. La décision finale d'adoption — positive ou négative — sera prise sur la base d'une évaluation structurée et documentée, pas sur la base de l'enthousiasme post-démonstration. C'est précisément ce que le SDLC réconcilié appelle la posture de discernement.

À retenir — Chapitre 4

- ONETOOL est un archétype — l'outil miracle qui promet d'automatiser le pipeline complet. Chaque génération technologique en produit un. La trajectoire typique : euphorie, friction, dépendance, paralysie.
- La Loi de la dépendance : plus un outil est puissant, plus la dépendance est difficile à reconnaître et plus le coût de sortie est élevé.
- Trois principes de relation saine avec tout outil : comprendre ce qu'il fait, maintenir la capacité à faire sans lui, avoir un plan de sortie viable.
- La question de Karim est la bonne : « Si cet outil est indisponible pendant une semaine, peut-on continuer à travailler ? Comment ? » Une hésitation dans la réponse est un signal d'information, pas une raison de refus automatique — mais elle doit être explorée.
- L'évaluation d'un outil doit couvrir cinq dimensions : périmètre réel,

atrophie induite, coût de sortie, transparence interne, dépendance organisationnelle.

- Le meilleur moment pour poser ces questions est avant l'adoption — jamais après, quand le coût de sortie rend la réponse honnête politiquement difficile.

Symptômes à observer — Chapitre 4

- L'équipe adopte régulièrement de nouveaux outils d'automatisation sans jamais en retirer. Le portfolio d'outils croît sans rationalisation.
- Une ou deux personnes seulement maîtrisent un outil critique. Leur absence crée immédiatement une zone de risque non gérée.
- La justification principale pour ne pas changer d'outil est « on est trop investis pour partir » — pas « cet outil est le meilleur choix pour notre contexte ».
- L'équipe ne peut pas répondre à la question : « Que faisions-nous avant cet outil, et serions-nous capables de le refaire ? »
- Les négociations de renouvellement de licence se déroulent sans levier de négociation, parce que tout le monde sait que partir n'est pas réellement envisageable.

Questions de réflexion — Chapitre 4

1. Quel est le ONETOOL de votre équipe en ce moment ? Depuis combien de temps est-il en place ? Avez-vous réévalué cette adoption

récemment ?

2. Si votre principal outil d'automatisation devenait indisponible demain pour une semaine, comment fonctionneriez-vous ? Avez-vous testé ce scénario ?

3. Quel est le coût de sortie réel de vos outils d'automatisation actuels ? Avez-vous une estimation honnête, ou évitez-vous de calculer ?

4. Y a-t-il des outils dans votre portfolio que vous maintenez par inertie plutôt que par conviction de leur valeur actuelle ?

5. Comment votre équipe structure-t-elle l'évaluation des nouveaux outils ? Existe-t-il un processus formel, ou chaque adoption est-elle une décision ad hoc ?

Signal de vigilance — Chapitre 4

Danger rouge : Votre équipe a adopté un outil d'automatisation dont le coût de sortie est si élevé — en termes techniques, humains, et financiers — que toute conversation sur une alternative se heurte à une résistance institutionnelle immédiate, avant même que les alternatives aient été évaluées. Ce n'est pas une décision stratégique d'outil — c'est une capture organisationnelle. La capture organisationnelle par un outil est l'une des formes de dette les plus difficiles à résoudre, parce qu'elle se défend elle-même en rendant la sortie de plus en plus coûteuse avec le temps.

CONCLUSION

Ce que la Partie A nous a appris — et ce qui attend en Partie B

La Partie A du Livre 13 a posé les fondations philosophiques et pratiques d'une réflexion sur l'automatisation qui, espérons-le, transcende les technologies et les contextes spécifiques. Avant de nous quitter pour la Partie B, il est utile de synthétiser ce que nous avons construit ensemble — et d'identifier ce qui reste à construire.

Ce que nous avons compris

Nous avons commencé dans une cave — avec un ingénieur passionné, un système MIDI impeccable, et un silence éloquent dans une salle de famille. L'anecdote de l'oncle MIDI n'est pas une métaphore de l'échec technologique. C'est une métaphore de l'automatisation sans intention claire. L'oncle MIDI n'a pas raté son système — son système était excellent. Il a raté la question préalable : que voulait-il vraiment produire ? Et cette question, nous l'avons vu, est celle que l'industrie du logiciel pose rarement avec la rigueur qu'elle mérite.

Nous avons exploré les cinq motivations profondes de l'automatisation — la Libération, la Précision, l'Échelle, la Sécurité, et la Curiosité. Chacune est légitime. Aucune n'est dangereuse en elle-même. Le danger naît du silence — de la motivation qui agit sans être nommée, qui guide des décisions d'investissement sans jamais être questionnée. La première discipline de l'automatisation responsable est de nommer sa motivation — honnêtement, sans honte — avant de commencer à construire.

Nous avons cartographié le SDLC réconcilié à travers le prisme de l'automatisation, et découvert une réalité surprenante : la zone ASSISTER — ni entièrement automatisée, ni entièrement humaine — représente environ 46% du SDLC de la plupart des équipes, et c'est la zone la moins bien adressée. C'est là

que les outils d'assistance intelligente, bien conçus et bien utilisés, ont le plus grand potentiel de valeur. Et c'est là que la réflexion de la Partie B se concentrera.

Nous avons nommé les quatre pièges — Automatiser le mauvais problème, La dette d'automatisation, L'atrophie des compétences, La fausse sécurité — et proposé pour chacun une règle ToTAL concrète. Ces pièges ne sont pas théoriques. Ils existent dans les équipes de toutes les tailles et de tous les secteurs. Les nommer ne suffit pas à les éviter — mais les ignorer garantit presque d'y tomber.

Et nous avons rencontré ONETOOL — non pas comme un outil spécifique, mais comme une posture : la tentation de déléguer le discernement à une solution qui promet de l'éliminer. La relation saine avec tout outil d'automatisation exige de comprendre ce qu'il fait, de maintenir la capacité à faire sans lui, et d'avoir un plan de sortie viable. Ces trois principes ne sont pas des obstacles à l'adoption — ce sont les conditions d'une adoption durable.

Ce que la Partie A ne résout pas

La Partie A a posé le diagnostic et établi le cadre de réflexion. Elle n'a pas résolu la question centrale du Livre 13 : *comment* construire une stratégie d'automatisation responsable, ancrée dans le SDLC réconcilié, qui libère réellement le meilleur de l'humain tout en exploitant pleinement la puissance de la machine ?

Cette question fera l'objet de la Partie B, qui abordera notamment :

- La construction d'une stratégie d'automatisation basée sur le risque — identifier les zones à automatiser en priorité en fonction de leur profil de risque, pas de leur facilité technique.
- Le maintien délibéré des compétences humaines en parallèle de l'automatisation — les pratiques concrètes, les rituels d'équipe, les indicateurs de vigilance.

- L'intégration de l'intelligence artificielle comme outil d'assistance dans la zone ASSISTER — avec ses possibilités réelles, ses limites réelles, et les conditions d'une relation saine.
- L'évaluation continue d'une automatisation existante : comment savoir si une automatisation remplit toujours son intention d'origine ? Quand la réviser, quand la déprécier, quand la remplacer ?
- La maturité d'automatisation d'une organisation — un modèle de progression qui distingue les organisations qui utilisent l'automatisation de celles qui en dépendent, et celles qui la maîtrisent.

La promesse fondamentale

La chaîne Clarté → qualité → confiance → excellence durable, qui est au cœur de la collection ToTAL — Réconciliation, n'est pas menacée par l'automatisation. Elle est amplifiée par une automatisation bien ciblée. Elle est fragilisée par une automatisation mal ciblée.

La Clarté — celle des spécifications, des intentions, des rôles — est le point de départ. Une automatisation construite sur des spécifications floues amplifie la confusion. Une automatisation construite sur des spécifications IF…THEN… rigoureuses amplifie la précision.

La Qualité — celle des tests, des revues, des critères de done — est amplifiée par l'automatisation déterministe et le test exploratoire complémentaire. Elle est dégradée par la fausse sécurité du pipeline vert interprété comme une déclaration de qualité suffisante.

La Confiance — celle des équipes entre elles, celle des utilisateurs dans le produit — est bâtie par la cohérence des livraisons, la fiabilité des systèmes, et la transparence des processus. Elle est érodée par les bugs en production que l'automatisation n'a pas vus parce que personne n'avait pensé à les tester.

L'Excellence durable, enfin, n'est pas l'automatisation maximale. C'est la combinaison optimale de l'intelligence humaine et de la puissance de la machine — chacune là où elle excelle, chacune respectée dans ses limites, chacune au service de la même intention : livrer des systèmes logiciels qui méritent la confiance de ceux qui les utilisent.

> *« L'automatisation bien ciblée libère le meilleur de l'humain. L'automatisation mal ciblée le remplace par le pire de la machine. »*
>
> — Léo Michel Bélanger, fondateur d'ORIGINALIUM

L'oncle MIDI, s'il lisait ces pages, reconnaîtrait peut-être que ce n'était pas son système MIDI qui avait échoué. C'était sa question. Et maintenant qu'il a la bonne question, il peut construire quelque chose de bien différent — quelque chose qui automatise ce qui mérite d'être automatisé, et qui préserve ce qui ne doit jamais l'être : la présence, la vulnérabilité, la joie de faire quelque chose pour quelqu'un qu'on aime.

Rendez-vous en Partie B.

Annexe A — Tableaux de synthèse de la Partie A

A.1 Les cinq motivations de l'automatisation — récapitulatif

Motivation	Intention noble	Piège associé	Question de calibration
1. Libération	Se libérer des tâches répétitives pour des activités à plus haute valeur	Libérer vers une activité moins définie encore	Vers quoi exactement cette libération mène-t-elle ?
2. Précision	Atteindre une précision impossible sur la durée pour un humain	Précision sur les mauvais critères	Mesure-t-on ce qui compte vraiment ?
3. Échelle	Faire à 1 000 ce qui ne peut se faire qu'à 1	Amplifier les erreurs de processus à l'échelle	Le processus est-il sain avant d'être amplifié ?
4. Sécurité	Éliminer les tâches épuisantes ou cognitivement risquées	Automatiser la vigilance elle-même	Qui reste responsable de la vigilance ?
5. Curiosité	Explorer le possible, innover, apprendre	Résoudre le mauvais problème avec élégance	Qui bénéficie de cette exploration ?

A.2 Cartographie de l'automatisation dans le SDLC réconcilié

Zone	Phase(s)	AUTOMATISER	ASSISTER	HUMAIN UNIQUEMENT
SENSE	ÉLICITER · DÉFINIR	Transcription, collecte de données brutes	Suggestion de cas de test, analyse de patterns, génération de gabarits	Élicitation des besoins, définition des spécifications, priorisation
FORGE	PLANIFIER ·	Tests	Revue de	Décisions

Zone	Phase(s)	AUTOMATISER	ASSISTER	HUMAIN UNIQUEMENT
	IMPLÉMENTER · DIFFUSER	unitaires, CI, CD, analyse statique, régression, alertes métriques	code assistée, suggestion architecturale, détection d'anomalies	architecturales, stratégie de test, investigation d'anomalies complexes
CARE	CONFIRMER → ÉLICITER	Collecte de métriques, génération de rapports, alertes de dérive	Analyse de tendances, suggestion de points de rétrospective	Interprétation qualimétrique, décision Go/No-Go, rétrospective, post-mortem

A.3 Les quatre pièges — règles ToTAL

Piège	Manifestation typique	Règle ToTAL
1. Mauvais problème	Automatisation d'un processus défaillant qui accélère la défaillance	Simplifier avant d'automatiser. Si le processus n'est pas ce qu'on voudrait de zéro, le reconstruire d'abord.
2. Dette d'automatisation	Scripts obsolètes, pipelines incompris, portfolio qui grandit sans rationalisation	Chaque ajout doit s'accompagner d'une politique de révision et dépréciation. Session trimestrielle de désherbage.
3. Atrophie	Équipe incapable de déployer manuellement quand le pipeline tombe en panne	On n'automatise jamais entièrement une compétence qu'on veut maintenir. Exercices manuels périodiques obligatoires.
4. Fausse sécurité	Pipeline vert = qualité déclarée, sans test	Les résultats d'automatisation sont

Piège	Manifestation typique	Règle ToTAL
	exploratoire complémentaire	un signal, jamais une conclusion. Le pipeline vert est une condition nécessaire, pas suffisante.

A.4 Grille d'évaluation d'un outil d'automatisation — résumé

Dimension	Question	Seuil de vigilance
Périmètre réel	Qu'automatise-t-il vraiment ?	Démonstration limitée aux cas simples
Atrophie induite	Quelles compétences disparaissent ?	Compétences critiques non maintenues
Coût de sortie	Que coûte de partir dans 12 mois ?	Réponse floue ou éludée
Transparence interne	Comprend-on ce qu'il fait ?	Boîte noire, logs cryptiques
Dépendance organisationnelle	Combien de personnes le maîtrisent ?	Une seule personne clé

Annexe B — Glossaire des termes clés de la Partie A

Terme	Définition dans le contexte de L13
SDLC réconcilié	Le cycle de vie du développement logiciel tel que défini par la collection

Terme	Définition dans le contexte de L13
	ToTAL — Réconciliation, organisé en six phases (ÉLICITER · DÉFINIR · PLANIFIER · IMPLÉMENTER · DIFFUSER · CONFIRMER) et trois zones (SENSE · FORGE · CARE), avec une grande boucle CONFIRMER → ÉLICITER.
Zone SENSE	Zone du SDLC réconcilié correspondant aux phases ÉLICITER et DÉFINIR. Zone de l'écoute, de la compréhension, et de la formalisation. Fondamentalement humaine.
Zone FORGE	Zone du SDLC réconcilié correspondant aux phases IMPLÉMENTER et DIFFUSER. Zone de la construction, de la validation, et du déploiement. Territoire naturel de l'automatisation.
Zone CARE	Zone du SDLC réconcilié correspondant à la phase CONFIRMER et à la grande boucle. Zone de l'évaluation, de l'apprentissage, et de la décision. Les données s'automatisent, l'interprétation reste humaine.
ONETOOL	Archétype de l'outil miracle qui promet d'automatiser un pipeline complet. Non péjoratif en soi — désigne tout outil dont l'adoption non discernée peut mener à une dépendance organisationnelle.
Motivation 5 (Curiosité)	Cinquième motivation d'automatisation — automatiser parce que le problème est techniquement fascinant, indépendamment du besoin réel. La plus répandue et la moins avouée dans l'industrie logicielle.
Dette d'automatisation	Coût différé des décisions d'automatisation qui sacrifient la maintenabilité, la pertinence, ou la compréhension collective à la vitesse

Terme	Définition dans le contexte de L13
	de construction. Souvent plus dangereuse que la dette technique car invisible.
Atrophie des compétences	Perte progressive de la capacité à exécuter manuellement ce qui a été entièrement automatisé. Risque organisationnel réel, non théorique.
Fausse sécurité	Confiance excessive dans les résultats d'automatisation, conduisant à interpréter le pipeline vert comme une déclaration de qualité suffisante, sans test exploratoire complémentaire.
Zone ASSISTER	Catégorie d'activités ni entièrement automatisables ni entièrement humaines, où la machine supporte le jugement humain. Représente environ 46% du SDLC réconcilié. Zone de plus grande valeur potentielle pour les outils d'assistance intelligente.
Test exploratoire	Activité de test où la conception et l'exécution sont simultanées, guidées par l'observation, l'intuition, et la curiosité du testeur. Par définition non automatisable. Complémentaire irremplaçable de l'automatisation.
Loi de la dépendance à l'outil	Principe ToTAL : plus un outil est puissant, plus la dépendance à cet outil est difficile à reconnaître, et plus le coût de sortie est élevé.
DREAM-TEAM ORIGINALIUM	Équipe fictive illustrant les dynamiques de la collection ToTAL. Composée de Rania (tech lead), Théo (Product Owner), et Karim (QA).

Pourquoi automatisons-nous ? La question paraît évidente. Dans les faits, l'industrie du logiciel l'a esquivée pendant des décennies — occupée à répondre avec une sophistication croissante à la question du *comment*, elle a négligé le *pourquoi*. Et c'est précisément de cette esquive que naissent les projets d'automatisation qui échouent, les pipelines qui deviennent ingérables, les équipes qui perdent leurs compétences fondamentales, et les systèmes qui passent tous leurs tests mais s'effondrent en production.

Le Livre 13 de la collection ToTAL — Réconciliation part d'une anecdote simple et fondatrice — celle de l'oncle MIDI — pour explorer les origines profondes de l'automatisation : ses cinq motivations fondamentales, ses quatre pièges classiques, ses zones d'excellence et ses zones d'exclusion dans le SDLC réconcilié, et la tentation permanente de l'outil miracle qui promet d'automatiser ce que seul le jugement humain peut résoudre.

C'est le livre le plus philosophique de la collection. Et peut-être le plus utile. Parce qu'avant de construire le bon pipeline, il faut se poser la bonne question. Et la bonne question, ce n'est pas « comment automatiser ». C'est « pourquoi ? Et vers quoi ? »

« L'automatisation bien ciblée libère le meilleur de l'humain. L'automatisation mal ciblée le remplace par le pire de la machine. »

Léo Michel Bélanger, fondateur d'ORIGINALIUM

Images et couverture par Soléane Bélanger
Collection ToTAL — Réconciliation · 21 livres
Livre 13 — Le genèsis humain de l'automatisation · Partie A